Prajwal Hebbar

Senior Full-Stack Engineer

prajwalhebbaras@gmail.com · +91 94839 24880 · Bengaluru, India · linkedin.com/in/prajwal-hebbar-a-s-a213b121a · github.com/prajwal-hebbar-07 · portfolio.hebbars.in

SUMMARY

Senior Full-Stack Engineer with nearly 5 years building configurable, enterprise-grade web platforms across **React front ends, Node.js APIs, SQL databases, auth and Docker/CI/CD**, with deep experience in schema-driven frontend architecture: admin dashboards, permission systems and rules/actions engines. Client-facing: gathers requirements directly from customers, aligns backend, QA and product, and **mentors juniors**. Promoted to Senior Engineer at LYIK in April 2026 after shipping v2 and driving much of the v3 form platform. Outside work I ship **local-first Tauri 2 desktop and Expo mobile apps** in TypeScript and Rust, release pipeline included.

EXPERIENCE

Senior Engineer — LYIK Technologies Pvt. Ltd. **Apr 2026 – Present**

On-site · Bengaluru · Previously Enterprise Engineer (Apr 2023 – Mar 2026)

- Owned a **regulatory compliance dashboard** for a financial-services client end to end — requirement gathering through architecture, build, production deployment and handover docs (React 19 · Vite 7 · Tailwind 4, Node API on `node:sqlite` with Zod-validated Excel ingestion, two-container Docker Compose behind nginx) — and **mentored the intern** who built its provider-comparison and data-exclusion features on top.
- **Top contributor on the v2 form-filling platform** (~330 commits), from project init to production: Redux Toolkit → **RTK Query** data layer, Tailwind→MUI migration, records and checker flows, DigiLocker OAuth2, liveness/KYC, e-sign and trusted APIs, shipped to **Azure** via GitHub Actions.
- Restructured **v3** into a **Turborepo/pnpm monorepo** — types, auth, API client, form core, transformer and shared UI as workspace packages — and built its **admin dashboard**: a config-driven widget renderer fed by `dashboard.json`, a form-management console (Material React Table) and a **user-management tree-table** modeling the org as a hierarchy/DAG with full CRUD.
- Built the persona/permission model behind both apps — **maker-checker** workflow state machine, `GuardRail` permission gating, bulk **CSV/ZIP onboarding**, trusted API records — and worked across the shared form **engines** (actions, rules, derive, navigator) with their unit, integration and e2e suites.
- Ran **customer requirement discussions** and drove alignment across backend, QA and product, turning the outcomes into the JSON configuration models the platform ships with.
- Delivered **SSO**, **client-side encryption**, **JSON-driven theming** and PWA/runtime config; set up **Docker/Nginx** and **GitHub Actions** CI/CD; built a **Frappe-based ERP** and a ServiceNow POC. Contributed to the company's **first enterprise client acquisition**.

STACK: React 19, TypeScript, Redux Toolkit, RTK Query, TanStack Query, Material UI, Material React Table, Tailwind CSS 4, Turborepo/pnpm, Node.js, SQLite, Zod, Vitest, Playwright, Docker, Nginx, GitHub Actions, Azure, Frappe

Associate Web Developer — Content Enablers Inc. **Mar 2022 – Mar 2023**

Remote · Bengaluru

- Led a **website revamp** using ReactJS and Bootstrap 5.
- Integrated **Google Analytics** and Google Tag Manager for product insight.

STACK: React, Bootstrap 5, Strapi CMS, Google Analytics, GTM

PROJECTS

Prompt Burn — local AI coding token dashboard **github.com/prajwal-hebbar-07/prompt-burn**

Tauri 2 · VS Code Extension · React · SQLite — Active build · shared local database

- Estimates what OMP and Cursor usage would cost if billed at **public pay-as-you-go rates** for comparison, never as an invoice; **provider usage clocks** stay on a dedicated panel.
- Desktop app and VS Code editor tab share **one local database** at `~/.prompt-burn/db.sqlite`, with no web app, account, or cloud backend.
- Usage is priced against a **bundled versioned rate table**; unknown models show an em dash (—) rather than `$0`, and retroactively price when added. Fetch is local and non-blocking.

STACK: Tauri 2, VS Code Extension, React, TypeScript, Tailwind CSS, SQLite, Node.js, pnpm workspace

anime-list — personal offline-first anime tracker **github.com/prajwal-hebbar-07/anime-list**

Tauri 2 · React 19 · Expo 57 · SQLite · Node.js — Active build · desktop, Android and sync

- Desktop and mobile clients operate **fully offline on local SQLite**, tracking catalogue entries, seasons, custom categories, and per-episode watch progress.
- Core schema, migrations, CRUD helpers, and sync protocol live in a **shared TypeScript package** consumed directly as source across all applications.
- Sync is **optional and self-hosted** (outbox + last-write-wins); optional **Ollama extraction** on the server turns public anime URLs into structured metadata and synopsis with cached posters.

STACK: Tauri 2, React 19, Expo 57, React Native, TypeScript, SQLite, Node.js, Ollama, Docker, pnpm workspace

LedgerFlow — local-first desktop expense tracker **github.com/prajwal-hebbar-07/ledger-flow**

Tauri 2 · React 19 · Rust · SQLite — shipping signed macOS/Linux/Windows releases

- Models every money movement as **one row** in a single table — spend, income, self-transfer and card charge differ only by `direction` and which source column is set; **balances are derived in SQL**, so no stored total can drift from the ledger.
- Analytics and a **rules-based report generator** over one period picker: period-over-period deltas and charts built from CSS boxes, not a charting library. Optional **Ollama** categorisation writes prose, never figures.
- Disk and network access live in **Rust commands**; **GitHub Actions** cuts macOS/Linux/Windows builds and a **minisign-signed** manifest drives in-app auto-update.

STACK: Tauri 2, React 19, TypeScript, Rust, SQLite, Tailwind CSS 4, Ollama, pnpm workspace, Turborepo, GitHub Actions

Flex State — offline gamified workout app + reactive store **github.com/prajwal-hebbar-07/flex-state**

pnpm + Turborepo monorepo · Tauri 2 · React 19 — app, UI package and published store

- **Deterministic plan generation**: identical profile, catalog and locations always yield an identical weekly plan, with equipment and exclusions resolved **per training ground**; the stamped generator version forces explicit regeneration.
- XP, level, rank and streak are **derived from the ordered completion history** — an `INSERT OR IGNORE` on the date key makes a repeat claim idempotent, and every persisted value passes a shape guard that fails closed.

- Published `flex-state`, a framework-agnostic store with `Object.is` dedupe, bound to React 19 through `useSyncExternalStore`; covered by Vitest and Node's test runner.

STACK: Tauri 2, React 19, TypeScript, Rust, SQLite, pnpm workspace, Turborepo, Vitest, Biome

GrowthOS — local-first study-plan tracker

private repository

Tauri 2 · React 19 · Expo 57 · Rust · SQLite — macOS desktop plus Android/iOS clients

- Two clients over **one model**: `@growth-os/learning` is consumed as TypeScript source with no build step, so the desktop app and the Expo React Native app share the same types, validators and eight bundled courses.
- Progress is **dated completions in SQLite** — one row per finished task, so totals and the consecutive-day streak derive from the ordered history and re-ticking never rewrites the original date.
- Task chat and day summaries call **Ollama from Rust**, never the webview, so no key reaches the frontend; summaries append into a markdown notes folder under a dated heading, and versioned **backup/restore** rejects a newer format whole rather than losing ticks.

STACK: Tauri 2, React 19, TypeScript, Rust, SQLite, Expo 57, React Native, Ollama, pnpm workspace, Turborepo

Portfolio — personal site with a grounded AI assistant

portfolio.hebbars.in · github.com/prajwal-hebbar-07/portfolio-hebbar

Astro · TypeScript · Go assistant API — static site, runtime personalization, Ask AI

- One content module is the **single source of truth** for the rendered sections, the resume and the assistant's grounding, so a fact is written once and cannot drift between the page and the answers.
- **Ask AI** posts through a same-origin Astro endpoint to a Go assistant API; when that backend is unreachable the endpoint serves a **keyword-routed grounded answer** instead of an error, and an off-topic prompt gets the same guardrail refusal either way. Prompts are validated and capped server-side.
- **Personalize panel** re-themes the site live — ground and primary colour, typeface and section order — over CSS design tokens, with the whole site prerendered static except the one assistant route.

STACK: Astro, TypeScript, CSS design tokens, Go API, Vercel

SKILLS

Frontend	React, Next.js, Astro, TypeScript, JavaScript, Redux Toolkit, TanStack Query, Material UI, Tailwind CSS, Design Tokens	**Backend, Data & Auth**	Node.js, Hono, REST APIs, PostgreSQL, SQLite, Rust (Tauri), OIDC/OAuth2, SSO, MFA, permission models, client-side encryption
DevOps & Platforms	Docker, Docker Compose, Nginx, GitHub Actions, Azure, Vercel, React Native/Expo, Tauri 2	**Architecture & Testing**	System design, schema-driven UI, Rule/Derive/Actions engines, Monorepo, Local-first apps, Vitest, Playwright
Delivery & Ownership	Requirement gathering, client communication, cross-team coordination, mentoring, production deployment, technical documentation		

EDUCATION

B.E. in Computer Science Engineering — VVCE, Mysuru — Visvesvaraya Technological University

Languages English · Kannada · Hindi